

Day 8 — API deployment, MLOps/LLMOps, security, and observability

Outcome

Be able to turn a model, RAG pipeline, or agent into a validated, secure, observable, scalable, versioned, evaluated, and reversible production service.

Scope note: Day 2 owns general HTTP, persistence, and concurrency semantics. This day applies those foundations specifically to ML, RAG, and agent release, serving, monitoring, security, and incident response.

1. Production API contracts

Prediction

```
POST /v1/predictions
```

Response includes prediction, model version, request ID, and safe metadata.

RAG

```
POST /v1/answers
```

Return answer, citations, request ID, and relevant component versions. Support insufficient-evidence behavior.

Ingestion

```
POST /v1/documents/{id}/ingestion
GET /v1/jobs/{job_id}
```

Return 202 Accepted; do parsing/chunking/embedding asynchronously.

Agent

```
POST /v1/agent-runs
GET /v1/agent-runs/{id}
POST /v1/agent-runs/{id}/approval
POST /v1/agent-runs/{id}/cancellation
```

Expose explicit states and approval requirements rather than holding a long HTTP request.

Boundary controls

- Pydantic/schema validation.
- Authentication and server-derived tenant.
- Resource/action authorization.
- Request, token, and concurrency limits.
- Idempotency for retried creations/actions.
- Timeouts and cancellation.
- Stable error structure.

- API and behavior versions.

Authentication is identity; authorization is permission. Apply authorization during retrieval and before tools.

2. FastAPI and serving patterns

Model in API process

Benefits:

- simple deployment;
- low internal network overhead.

Costs:

- each worker may load a large model;
- memory and scaling are coupled;
- startup and worker-count mistakes are expensive.

Separate API and model server

Benefits:

- independent scaling;
- shared accelerators/batching;
- specialized serving runtime;
- provider/model routing.

Costs:

- another network dependency;
- contracts, retries, and observability required.

Do not load a model for every request.

Health endpoints

- Liveness: should process restart?
- Readiness: may this instance receive traffic?
- Startup: has slow initialization completed?

Do not make liveness depend directly on a brief third-party model outage. Readiness should represent the service's ability to handle its contract.

3. Sync, async, batch, streaming, and edge

- Real-time: interactive, latency-sensitive inference.
- Batch: large scheduled/offline scoring and embedding.
- Streaming: continuous event predictions or token/event output.
- Edge: inference near device/operation under local constraints.

Async web programming overlaps I/O but does not accelerate CPU/GPU inference by itself. Long work belongs in workers/queues or a durable runtime.

4. MLOps and LLMOps lifecycle

DevOps manages software build, deployment, reliability, and operations. MLOps adds data, features, experiments, model registry, drift, and retraining. LLMOps additionally

tracks prompts, retrieval/index configuration, traces, evaluations, tools/agents, safety, tokens, and cost.

```
data/version
→ train/build
→ evaluate
→ register/package
→ deploy gradually
→ monitor
→ retrain/change/rollback
```

CI tests code, data contracts, model/RAG behavior, safety, APIs, and packaging.

CD promotes an approved immutable model/application/configuration set.

Test layers

- Unit tests.
- Data/schema tests.
- Model-quality baseline tests.
- Bias/fairness tests.
- Robustness/adversarial tests.
- API/contract tests.
- RAG golden tests.
- Tool trajectory and side-effect tests.
- Failure/recovery tests.

Release flow

```
offline evaluation
→ shadow traffic
→ small canary/challenger
→ compare quality, errors, latency, cost, safety
→ gradual rollout
→ champion/full release
```

Version together when behavior depends on them:

- model;
- prompt;
- embedding model;
- parser/chunker;
- index;
- retrieval/reranker settings;
- tool schemas;
- agent graph;
- guardrail policy.

Rollback must have a known stable target.

Control plane versus data plane

The data plane handles live retrieval, models, tools, and streaming.

The control plane manages tenant setup, configuration, component versions, evaluation datasets, policies, rollout, and rollback.

Separating them prevents live request code from becoming the only place to govern prompts, models, indices, tools, and graph versions.

5. Drift and monitoring

- Data drift: input distribution changes.
- Concept drift: relationship between inputs and outcomes changes.
- Model-performance drift: measured predictive behavior deteriorates.

Infrastructure

- CPU/GPU/memory;
- queue depth/age;
- connection pools;
- pod restarts;
- network/storage/database latency.

Application

- request/error/timeout rate;
- p50/p95/p99 latency;
- dependency latency;
- retries and circuit state;
- cache hit;
- concurrency/admission rejection.

ML

- prediction distribution;
- labeled accuracy/precision/recall/F1 when available;
- drift;
- calibration/fairness where required;
- business outcome.

RAG

- retrieval and reranker latency;
- Recall@k/Precision@k;
- empty retrieval;
- citations and groundedness;
- freshness/ACL failures.

Agent

- tool calls and success;
- steps, loops, retries, approval;
- completion/escalation;
- policy denial;
- cost per completed task.

Trace:

```
request
→ retrieval
→ reranking
→ prompt/model
→ tool/approval
→ response
```

Use safe identifiers and versions, not unrestricted content.

6. Reliability patterns

Pattern	Use
Timeout	Bound every dependency.
Retry with backoff/jitter	Known transient, idempotent failures.
Circuit breaker	Stop amplifying a failing provider.
Bulkhead	Isolate chat, ingestion, tool, or tenant pools.
Backpressure	Slow intake as queues/capacity fill.
Admission control	Reject early by quota/capacity.
Idempotency	Prevent repeated effects.
Dead-letter queue	Retain exhausted jobs for investigation/replay.
Graceful degradation	Use a safe reduced path.
Reconciliation	Resolve stuck/unknown state.

Avoid retry amplification across client, API, service mesh, worker, and SDK.

7. Inference and cost

Provider API versus self-hosting

Managed provider:

- fast adoption and managed scale;
- ongoing token cost, rate limits, network/vendor/governance constraints.

Self-hosted:

- control, locality, customization;
- GPU planning, serving, security, scaling, patching, and on-call burden.

Compare total cost of ownership.

Serving runtimes

The source notes position:

- Ollama as convenient local/developer runtime.
- vLLM and TGI as production-oriented text-generation serving choices.

Benchmark your model, hardware, context lengths, and concurrency.

Batching

Improves accelerator utilization and throughput; waiting for batches and variable sequence lengths add latency/memory trade-offs.

Quantization

Lower-precision weights reduce memory and can improve throughput, with possible quality and compatibility costs. Evaluate end-to-end task quality.

Caching

- prompt-prefix;
- response;
- retrieval;
- embedding/reranking.

Keys include tenant/access, component versions, input, and relevant model settings. Private results cannot cross scopes.

Streaming

Improves perceived latency and supports agent events. Handle partial errors, cancellation, backpressure, moderation, billing, tracing, and structured-output reconstruction.

REST versus gRPC

REST is browser-friendly and easy to debug. gRPC offers strong typed contracts, efficient internal transport, and streaming. A platform may expose REST externally and gRPC internally.

8. Security, privacy, and tenant isolation

Identity and access

Authentication options in the source notes include password/session, scoped API keys, OAuth, enterprise SSO/OpenID Connect-style identity, bearer/JWT tokens, and client certificates. Match the method to user, service, and delegated-access needs.

API keys need hashing where appropriate, rotation, scopes, expiration, and usage tracking. OAuth is delegated authorization; OpenID Connect supplies an identity layer. Successful authentication never grants universal access.

Validate JWT signature, issuer, audience, expiration, algorithm, token type, roles/scopes, and tenant claims.

OAuth provides delegated access; give only required scopes.

RBAC can be combined with tenant, ownership, department, geography, classification, purpose, and relationship.

Rate limiting

Limit by IP, user, key, tenant, endpoint, model, token use, and concurrent work. Request count alone misses expensive LLM calls.

Algorithms include fixed window, sliding window, token bucket, and leaky bucket. Select a policy that matches burst tolerance and fairness.

WAF and DDoS

A WAF can block or challenge known malicious patterns, suspicious bots, abnormal request shapes, oversized payloads, and common web attacks.

DDoS controls include edge protection, rate limits, request-size limits, load shedding, autoscaling, admission control, and circuit breakers. A WAF does not understand every semantic prompt attack.

Encryption and secrets

- TLS in transit.
- Encrypt databases, objects, indexes, logs, datasets, and backups.
- Keep secrets out of code, images, prompts, and logs.
- Use least-privilege workload identities.

Encryption does not replace authorization.

PII

Control before model call, before logging, during ingestion, before external routing, and after output. Use classification, redaction, tokenization, or PII masking where appropriate. Apply purpose limitation; some authorized workflows genuinely require PII.

Prompt injection

Untrusted user/retrieved/tool content may instruct the model to ignore policy or exfiltrate data.

Controls:

- clear instruction/data separation;
- minimal tools/credentials;
- egress allowlists;
- server-side authorization;
- schema/business validation;
- approval;
- content classification;
- sandboxing where applicable;
- anomaly monitoring.

No single prompt filter is complete.

Data poisoning

An attacker or faulty source can place manipulated content into training data or the knowledge base so future model/retrieval behavior is corrupted.

Controls include source trust and ownership, ingestion validation, hashes/versioning, anomaly and quality checks, approval before index/model publication, lineage, evaluation against reviewed cases, and rollback to a known data/index/model version.

Jailbreaks and exfiltration

Even a jailbroken model must not possess unrestricted credentials or network access.

Watch:

- unauthorized tool results;

- external URLs;
- cross-tenant cache/index errors;
- log leakage;
- prompt secrets;
- malicious retrieved content.

Output filtering

Risk-based output checks can cover:

- PII or credentials;
- toxicity or disallowed advice;
- unsupported factual claims;
- malicious links, code, or commands;
- schema validity.

False positives can reduce utility. Apply controls proportionally to the use case, preserve safe refusal/escalation paths, and do not mistake output filtering for authorization at data/tool boundaries.

Tenant isolation choices

1. Shared index with mandatory tenant/ACL filters: lowest cost, filter failure risk.
2. Namespace/collection per tenant: stronger logical isolation, more operations.
3. Separate indices/infrastructure: strongest isolation, highest cost.

Choose tiers based on sensitivity, regulation, scale, contracts, and operational cost. Test cross-tenant attacks continuously.

9. Governance and responsible AI

Govern:

- approved data/models/tools;
- ownership and lineage;
- evaluation and promotion evidence;
- bias/fairness/safety;
- access and approvals;
- retention/deletion;
- audit;
- incident and rollback process.

Responsible AI principles in the source material include fairness, transparency/explainability, privacy, safety, accountability, and human oversight. Translate them into evaluated requirements, ownership, approvals, monitoring, and audit rather than leaving them as statements.

Audit must answer who, what version, which data/tool, what approval, what outcome, and when—without becoming a second unprotected sensitive-data store.

Operational response and learning

A production design also needs a response loop:

alert or failed evaluation

- identify affected version, tenant/scope, and dependency
- contain through denial, fallback, traffic reduction, or rollback
- reconcile unknown tool/job outcomes
- preserve safe evidence for audit
- correct the root cause
- add the case to tests, evaluations, or runbooks

Maintain ownership, alert thresholds, runbooks, rollback criteria, and a review process for failed evaluations and incidents. Restoring service is not the end: confirm data/index/tool consistency and feed validated lessons back into release gates.

10. Production readiness checklist

API:

- validated contracts, stable errors, idempotency, versioning, pagination, async job states.

Model/RAG/agent:

- approved versions, golden tests, citations, bounds, safe tools, fallbacks.

Security:

- authn/authz, tenant filtering, least privilege, secrets, encryption, safe logs, rate/token limits.

Reliability:

- timeouts, bounded retries, circuit breakers, queues, DLQ, reconciliation, rollback.

Observability:

- metrics/logs/traces across infrastructure, application, AI quality, and business.

Deployment:

- immutable artifact, staged rollout, health/quality gates, known rollback.

Project-grounded examples

Scenario 1: CI quality gates for a production-style RAG/agent service

Project scenario. **DPDK BenchOps Copilot** was exposed through FastAPI and deployed using Kubernetes and Helm with HPA and Jenkins. Its release checks included faithfulness/groundedness, context precision, context recall, citation coverage, tool success rate, tool error rate, and p95 latency. The platform also traced retrieval and tool calls and used retries, timeouts, circuit-breaker-style dependency protection, and canary/rollback thinking.

How the concepts apply. This is LLMOps rather than ordinary “service is up” monitoring. A release can pass unit tests and still retrieve the wrong workload context, lose citations, call tools less reliably, or exceed its latency target. The project therefore treated retrieval behavior, answer support, tool outcomes, and latency as release dimensions.

Decision and trade-offs. Golden-set gates increased CI time and required maintained evaluation cases, but prevented prompt, retrieval, or tool changes from silently degrading operational quality. Kubernetes/HPA enabled production-style scaling, while

the multi-component service increased dependency and observability complexity. Canary/rollback reduced change risk but required compatible application, retrieval, prompt, tool, and data versions.

Senior/Staff interview framing.

- **Senior:** describe one release change, the deterministic tests and AI evaluations it should run, the metrics that would block promotion, and the rollback artifact.
- **Staff:** define the quality contract and ownership model across code, prompt, index, tools, and deployment. Explain how you balance release speed with evaluation cost, and how online failures become reviewed golden cases.

Evidence boundary. The project does not record exact gate thresholds, traffic volume, autoscaling signals, image-registry details, or measured availability. Do not invent them.

Scenario 2: architectural safety for commands and disruptive actions

Project scenario. The original DPDK platform contained complex command templates and BIOS automation, including Redfish-based changes for Dell/HP and Python automation for AMD Cinnabar platforms. In the Copilot, `CommandBuilder` generated commands only from allowlisted templates; run access, log retrieval, and comparisons were exposed as narrow MCP tools; calls were audited; and BIOS/reboot-affecting actions required human control.

How the concepts apply. The model had no need for arbitrary shell access. Prompt instructions were supplemented by hard capability boundaries: deterministic tool implementations, constrained command templates, verification, audit, and approval for the highest-impact operations.

Decision and trade-offs. An allowlist cannot express every novel command an expert might want, so it trades flexibility for controlled behavior. Manual BIOS approval adds delay, but limits a disruptive and platform-sensitive risk. Audit and verification add storage and latency, but support incident review and operator trust.

Outcome. The project reports safer, more reproducible operational workflows and auditable tool use. It does not claim zero incidents or quantify risk reduction.

Senior/Staff interview framing.

- **Senior:** show validation before a `CommandBuilder` or run-access call, safe failure behavior, and the data captured in an audit event.
- **Staff:** present a risk-tiered capability model—read, deterministic preparation, and disruptive execution—with progressively stronger policy and approval. Connect it to least privilege, incident response, and release evaluation.

Security scope boundary. The project documents tool controls and approvals, but not multi-tenant isolation, OAuth/OIDC, JWT validation, WAF/DDoS configuration, PII processing, encryption implementation, or a secrets product. Discuss those as general production requirements or clearly labeled **hypothetical improvements**, not completed project work.

11. Interview questions

1. Why is input validation not authorization?
2. Why can `async` fail to speed CPU-heavy inference?
3. Model inside API or separate model server?
4. What belongs in a RAG or agent API response?

5. Data drift versus concept drift versus measured model-performance degradation?
6. What should block an AI release?
7. Provider API versus self-hosting?
8. What do batching, quantization, caching, and streaming trade?
9. What should be monitored for ML, RAG, and agents?
- 10
 - . How do you avoid retry amplification?
- 11
 - . How do you defend against prompt injection?
- 12
 - . How do tenant-isolation options trade cost and risk?
- 13
 - . Why is safe logging difficult in GenAI?
- 14
 - . What makes rollback complete for a RAG/agent system?
- 15
 - . What belongs in the control plane versus data plane?
- 16
 - . Why are WAF/DDoS controls different from prompt-injection controls?
- 17
 - . How would you detect and contain data poisoning?
- 18
 - . After rollback, what reconciliation and learning work remains?

12. Exit checklist

- [] Design prediction, RAG, ingestion, and agent APIs.
- [] Choose serving/deployment patterns.
- [] Build CI/CD evaluation and rollback gates.
- [] Explain drift and all telemetry layers.
- [] Apply reliability patterns without uncontrolled retries.
- [] Compare hosted/self-hosted inference and optimization levers.
- [] Enforce identity, access, tenant scope, PII, and tool safety.
- [] Define audit/governance/responsible-AI controls.
- [] Define incident ownership, runbooks, containment, reconciliation, and post-incident evaluation updates.

Source notes

- [MLOps for AI Systems](#)
- [Deploying ML Models API](#)
- [Cloud AI Architecture](#)
- [Enterprise GenAI Solution Design](#)
- [Python Core](#)
- [Python Advanced](#)
- [Async and Concurrency](#)
- [Vanilla RAG](#)
- [LangGraph End to End](#)

- MCP End to End
- Capstone Revision Day 2
- Capstone Revision Day 3
- DPDK Automation for Network Packet Processing
- DPDK BenchOps Copilot